

گزارش پروژه

سامانه تحلیل داده‌های فروش و نمایش گزارش‌های آماری

Sales Analytics Platform

نام دانشجو	مبین ایچی
نام استاد	آزیتا شیرازی پور
نام درس / واحد	پروژه نرم افزار
دانشگاه / مؤسسه	آزاد تهران شرق
تاریخ تحویل	

React + TypeScript + FastAPI + PostgreSQL: فناوری‌های اصلی

فهرست مطالب

- ۱. عنوان و معرفی پروژه
- ۲. معرفی مسئله و اهداف
- ۳. نیازمندی‌های سیستم
- ۴. کاربران سیستم و نقش‌ها
- ۵. امکانات نرم‌افزار
- ۶. تکنولوژی‌های استفاده‌شده
- ۷. معماری نرم‌افزار
- ۸. توضیح ماژول‌ها
- ۹. طراحی پایگاه داده
- ۱۰. API و جریان تبادل داده
- ۱۱. مشکلات پیاده‌سازی و راه‌حل‌ها
- ۱۲. راه‌اندازی و اجرای پروژه
- ۱۳. آزمون و ارزیابی
- ۱۴. توسعه‌های آینده
- ۱۵. نتیجه‌گیری

۱ عنوان و معرفی پروژه

عنوان	مقدار
نام پروژه	سامانه تحلیل داده‌های فروش و نمایش گزارش‌های آماری
نام انگلیسی	Sales Analytics Platform
نوع نرم‌افزار	Business Intelligence / داشبورد مدیریتی / Analytics SaaS
فرانت‌اند	React + Vite + TypeScript
بک‌اند	FastAPI
پایگاه داده	PostgreSQL
معماری ارتباط	REST API / api/v1 با نسخه‌بندی

سامانه تحلیل داده‌های فروش یک نرم‌افزار تحت وب برای تجمیع، نمایش و تحلیل اطلاعات فروش است. هدف اصلی سامانه این است که مدیران و تحلیل‌گران بتوانند بدون بررسی دستی حجم زیادی از داده‌ها، وضعیت درآمد، سفارش‌ها، مشتریان، محصولات و مناطق جغرافیایی را در قالب شاخص‌های کلیدی، نمودارها و گزارش‌های قابل فهم مشاهده کنند.

۲ معرفی مسئله و اهداف

بیان مسئله

در بسیاری از کسب‌وکارها اطلاعات فروش در منابع مختلف نگهداری می‌شود و استخراج یک تصویر دقیق از عملکرد کسب‌وکار نیازمند بررسی فایل‌ها، گزارش‌های جداگانه یا محاسبات دستی است. این پراکندگی باعث افزایش زمان تحلیل، احتمال خطای انسانی و دشواری در مقایسه دوره‌های زمانی مختلف می‌شود. سامانه حاضر با تمرکز بر یک داشبورد متمرکز، داده‌های فروش را به اطلاعات مدیریتی قابل استفاده تبدیل می‌کند.

اهداف اصلی

- های خوانا و قابل مقایسه KPI نمایش وضعیت کلی فروش در یک نگاه با
- تحلیل روند فروش در بازه‌های زمانی ۷، ۳۰ و ۹۰ روزه و بازه‌های سفارشی در توسعه‌های بعدی
- بررسی عملکرد محصولات، موجودی و دسته‌بندی‌ها
- تحلیل مشتریان، بخش‌بندی مشتریان و شاخص‌های بازگشت مشتری
- نمایش وضعیت سفارش‌ها، فیلتر و جستجو در سفارش‌ها
- تحلیل جغرافیایی فروش بر اساس شهر یا منطقه
- تولید گزارش‌های مدیریتی و خروجی‌های قابل استفاده
- AI Insights، Forecasting و Anomaly Detection ایجاد بستری قابل توسعه برای
- توسعه یابند UI مستقل از Database و Backend جدا کردن کامل رابط کاربری از منبع داده، به گونه‌ای که

۳ نیازمندی‌های سیستم

نیازمندی‌های عملکردی

نیازمندی	شرح
احراز هویت	و خروج از حساب Refresh Token، ثبت‌نام، ورود، دریافت کاربر جاری
داشبورد	نمایش درآمد، سفارش‌ها، مشتریان، سود، نمودار فروش، محصولات برتر و سفارش‌های اخیر
محصولات	نمایش، جستجو، صفحه‌بندی، ایجاد، ویرایش و حذف متناسب با نقش کاربر
مشتریان	لیست مشتریان، خلاصه آماری و بخش‌بندی مشتریان
سفارش‌ها	جستجو و فیلتر سفارش‌ها، مشاهده وضعیت و مبلغ، ایجاد یا ویرایش در سطح مجاز
تحلیل فروش	روند فروش، مقایسه دوره‌ها و توزیع فروش
جغرافیا	نمایش فروش بر اساس شهر/منطقه و رتبه‌بندی مناطق
گزارش‌ها	در رابط کاربری Print/PDF و قابلیت CSV/JSON تولید گزارش و خروجی
جستجوی سراسری	Backend جستجو میان صفحات و داده‌های مهم سامانه از طریق
اعلان‌ها	نمایش اعلان‌ها، علامت‌گذاری به‌عنوان خوانده‌شده و مشاهده تعداد اعلان‌های جدید
تنظیمات رابط	Sidebar تم روشن/تاریک، اندازه فونت، حالت فشرده، کاهش انیمیشن و وضعیت

نیازمندی‌های غیرعملکردی

- Desktop، Laptop، Tablet و Mobile برای Responsive رابط کاربری
- برای کنترل مرکزی فونت و اندازه‌ها Design Token خوانایی مناسب تایپوگرافی و استفاده از
- Plain Text و عدم ذخیره رمز به‌صورت Hash امنیت رمز عبور با
- برای ادامه نشست کاربر Refresh Token برای احراز هویت و JWT استفاده از
- برای نگهداری‌پذیری Repository و Service و Router جداسازی لایه‌های
- پایگاه داده Schema برای تغییرات Migration استفاده از
- React به‌جای صفحه خام خطای Error State مدیریت خطاها و نمایش
- Dynamic Import و Lazy Loading بهینه‌سازی بارگذاری بخش‌های سنگین با

پیش‌نیازهای اجرا

مؤلفه	پیشنهاد
سیستم عامل	Windows 10/11، Linux یا macOS
Python	نسخه 3.12 یا بالاتر
Node.js	جدید LTS نسخه
PostgreSQL	نسخه 15 یا 16
مرورگر	جدید Chrome / Edge / Firefox
RAM حافظه	یا بیشتر برای توسعه GB: پیشنهاد GB8 حداقل 4
ابزار اختیاری	Docker و Docker Compose

۴ کاربران سیستم و نقش‌ها

بررسی می‌شود و Backend استفاده می‌کند. نقش کاربر در (Role-Based Access Control) سامانه از کنترل دسترسی مبتنی بر نقش فرانت‌اند نیز می‌تواند بر همان اساس دکمه‌ها و عملیات مجاز را نمایش دهد.

نقش	سطح دسترسی پیشنهادی
Admin	دسترسی کامل؛ مدیریت محصولات، مشتریان، سفارش‌ها، کاربران، گزارش‌ها و تنظیمات مدیریتی
Manager	مشاهده همه داده‌های مدیریتی، ایجاد و ویرایش داده‌های عملیاتی و دسترسی به گزارش‌ها
Analyst	مشاهده داده‌ها، تحلیل‌ها، داشبورد و گزارش‌ها؛ بدون عملیات مدیریتی حساس
Viewer	فقط مشاهده اطلاعات و گزارش‌ها؛ بدون ایجاد، ویرایش یا حذف داده‌های اصلی

۵ امکانات نرم‌افزار

داشبورد مدیریتی

- های درآمد، تعداد سفارش، تعداد مشتری، سود و نرخ رشد KPI
- نمودار روند فروش و مقایسه دوره زمانی
- نمایش محصولات برتر و سفارش‌های اخیر
- Dashboard برای نمایش بصری داده‌ها در صفحه Data City صحنه سه‌بعدی
- انتخاب بازه ۷، ۳۰ و ۹۰ روزه و اثرگذاری آن بر نمودارها و نمای سه‌بعدی

تجربه کاربری و ناوبری

- باز و جمع‌شونده در دسکتاپ و لپ‌تاپ Sidebar

- Overlay واقعی در موبایل همراه Drawer.
- Hamburger Menu برای باز/بسته کردن Sidebar.
- Ctrl+K جستجوی سراسری با میانبر.
- تم روشن و تاریک با ذخیره تنظیمات در مرورگر.
- پروفایل کاربر و منوی حساب.
- Notification Center و شمارنده اعلان‌های خوانده نشده.
- Typography System مرکزی برای کنترل اندازه و خانواده فونت از یک محل.

امنیت و نشست کاربر

- Backend از طریق Login و Register.
- Access Token برای درخواست‌های محافظت شده.
- Refresh Token برای تمدید نشست.
- Logout و Revoke کردن Refresh Token.
- Hash رمز عبور با Argon2.
- Backend های حساس API های فرانت و Route محافظت.

۶ تکنولوژی‌های استفاده شده

لایه	فناوری	کاربرد
Frontend	React	ساخت رابط کاربری کامپوننت‌محور.
Frontend	Vite	سریع Build و Dev Server.
Frontend	TypeScript	افزایش ایمنی نوع داده و نگهداری پذیری.
Frontend	React Router	Route Guard مدیریت مسیرهای صفحات و
Frontend	Chart.js	رسم نمودارهای آماری.
Frontend	Three.js	Dashboard نمایش المان‌های سه بعدی.
Frontend	Axios	JWT های Interceptor و مدیریت Request ارسال.
Backend	FastAPI	خودکار Swagger با REST API پیاده سازی.
Backend	Pydantic	داده‌های ورودی و خروجی Validation.
Backend	SQLAlchemy 2.x	ORM و ارتباط با PostgreSQL.
Backend	Alembic	های دیتابیس Migration مدیریت.
Database	PostgreSQL	ذخیره پایدار داده‌های سامانه.
Security	JWT + PyJWT	Access و Refresh Token.

امن رمزهای عبور Hash	Argon2	Security
Auth. ها و Endpoint تست	Pytest + HTTPX	Testing
API و PostgreSQL اجرای یکپارچه	Docker Compose	Deployment

۷ معماری نرم افزار

معماری سطح بالا

```

React / TypeScript
↓ HTTPS / REST
Axios + JWT Interceptors
↓
FastAPI /api/v1
↓
Service Layer
↓
Repository Layer
↓
SQLAlchemy 2.x
↓
PostgreSQL

```

وابستگی مستقیمی به نحوه ذخیره سازی داده ها نداشته باشد. در نتیجه تغییر منبع داده یا API این معماری باعث می شود فرانت اند به جز قرارداد امکان پذیر است React ها بدون بازنویسی صفحات Repository توسعه

معماری فرانت اند

```

src/
├── app/           Router و Provider ها
├── components/    UI, Chart و Navigation
├── layouts/       Sidebar, Navbar و DashboardLayout
├── pages/         صفحات اصلی
├── context/       UI/Auth State
├── services/      و سرویس های داده API Client
├── hooks/         های قابل استفاده مجدد Hook
├── types/         های TypeScript
└── styles/        Design Tokens و Typography

```

معماری بک‌اند

```

app/
├── api/v1/endpoints/ Request و Response دریافت
├── services/ Business Logic و Analytics
│   ├── repositories/ Query و دسترسی به داده
│   ├── models/ SQLAlchemy Models
│   ├── schemas/ Pydantic Schemas
│   └── database/ Session / Base / Init
├── core/ Config و Security
└── tests/ API و Auth تست‌های
    
```

جریان احراز هویت

```

Login → Verify Password → Access Token + Refresh Token
↓
React → Authorization: Bearer ACCESS_TOKEN
↓
FastAPI → Validate JWT → Current User → Protected Endpoint

Access Token Expired → Refresh Token → New Access Token
    
```

۸ توضیح ماژول‌ها

ماژول	مسئولیت
Auth	مسئول کنترل هویت و نقش کاربر است. Logout و Refresh Token، ثبت‌نام، ورود، دریافت مشخصات کاربر
Dashboard	ها، نمودار فروش، محصولات برتر و سفارش‌های اخیر KPI محاسبه و نمایش
Sales Analytics	روند فروش، خلاصه فروش، مقایسه دوره‌ها و توزیع دسته‌بندی‌ها
Products	محصولات، جستجو، صفحه‌بندی، موجودی، قیمت، دسته‌بندی و عملکرد محصول CRUD
Customers	Segmentation لیست مشتریان، اطلاعات تماس، شهر، خلاصه آماری و
Orders	نمایش، فیلتر و مدیریت سفارش‌ها، وضعیت، مبلغ، مشتری، شهر و تاریخ

Geography	تجميع فروش بر اساس شهر يا منطقه و نمايش مناطق برتر
Reports	تعريف و توليد گزارش و خروجی داده برای استفاده مدیریتی
Insights	در نسخه‌های بعدی AI/ML و بستر آماده برای MVP در Rule-Based های Insight
Notifications	اعلان‌های کاربر، وضعیت خوانده‌شدن و شمارنده اعلان جدید
Global Search	جستجوی سراسری بین موجودیت‌ها و مسیرهای مهم سامانه
Settings	؛ این موارد در سمت کاربر ذخیره می‌شوند Motion و Theme. Typography. Sidebar شامل UI تنظیمات

۹ طراحی پایگاه داده

بر اساس مدل رابطه‌ای طراحی شده است. جداول اصلی اطلاعات کاربران، مشتریان، محصولات و سفارش‌ها را PostgreSQL پایگاه داده اعلان‌ها، پرداخت‌ها و اهداف فروش استفاده می‌شوند. Refresh Token نگهداری می‌کنند و جداول جانبی برای

جداول اصلی

کاربرد	فیلدهای کلیدی	جدول
اطلاعات کاربران و نقش‌ها	id, email, password_hash, full_name, role, is_active, created_at	users
Refresh Token مدیریت نشست و	id, user_id, token_hash, expires_at, revoked_at	refresh_tokens
دسته‌بندی محصولات	id, name	categories
اطلاعات محصولات و موجودی	id, name, sku, price, stock, category_id	products
اطلاعات مشتریان	id, full_name, email, phone, city, created_at	customers
سرآیند سفارش	id, customer_id, status, total_amount, city, created_at	orders
اقلام هر سفارش	id, order_id, product_id, quantity, unit_price	order_items
اطلاعات پرداخت	id, order_id, method, status, amount	payments
اطلاعات جغرافیایی	id, name, city	regions
اهداف فروش	id, period, target_amount	sales_targets
اعلان‌های کاربران	id, user_id, title, message, is_read, created_at	notifications

روابط اصلی

Customer 1 — N Order
 Order 1 — N OrderItem
 Product 1 — N OrderItem
 Category 1 — N Product
 Order 1 — N Payment
 User 1 — N RefreshToken
 User 1 — N Notification

Migration و Seed

Backend اولیه که در مرحله طراحی فرانت استفاده می‌شدند، در Fake انجام می‌شود. داده‌های Alembic با Schema ساخت و تغییر Backend منتقل می‌شوند. در زمان اجرای واقعی PostgreSQL نگهداری می‌شوند و با اسکریپت مهاجرت به Seed Data به‌عنوان منبع عملیاتی سامانه نیست JSON می‌خوانند و فایل PostgreSQL ها اطلاعات را از Repository

۱۰ API و جریان تبادل داده

قابل مشاهده docs/ در مسیر FastAPI توسط Swagger. قرار دارند /api/v1 ها از ابتدا نسخه‌بندی شده‌اند و زیر مسیر API تمام است.

Endpoint های اصلی	دامنه
POST /auth/register. POST /auth/login. POST /auth/refresh. GET /auth/me. POST /auth/logout	Auth
GET /dashboard/summary. /sales-chart. /top-products. /recent-orders	Dashboard
GET /sales. /summary. /trend. /comparison. /category-distribution	Sales
GET/POST /products. GET/PATCH/DELETE /products/{id}	Products
GET/POST /customers. GET/PATCH /customers/{id}. /summary. /segments	Customers
GET/POST /orders. GET/PATCH /orders/{id}	Orders
GET /geography/sales. /cities. /regions	Geography
GET /reports. POST /reports/generate	Reports
GET /insights	Insights
GET/PATCH اعلان‌های کاربر	Notifications
GET جستجوی سراسری	Search

Frontend نمونه تنظیم اتصال

```
VITE_API_URL=http://localhost:8000/api/v1
```

درخواست‌های Header را به Access Token، احراز هویت Interceptor استفاده می‌کند و Base URL از این Axios Client محافظت شده اضافه می‌کند.

۱۱ مشکلات پیاده‌سازی و راه‌حل‌ها

مشکل	راه‌حل
در فرانت Fake Data پراکندگی	با JSON های Repository منتقل شدند؛ سپس Backend حذف و ابتدا به React داده‌ها از Repository های PostgreSQL جایگزین شدند
Dashboard کندی بارگذاری صفحه	Mock Service مصنوعی Delay بارگذاری شدند و Lazy/Dynamic به صورت Chart.js و Three.js حذف شد
Rendered more hooks than during the previous render خطای	ها در تمام Hook های شرطی قرار گرفتند تا ترتیب Return قبل از useMemo هایی مانند Hook ها ثابت باشد
Collapse در حالت Sidebar مشکل	مخفی می‌شود و آیکن باقی Label فقط Collapse به عناصر مستقل تبدیل شدند؛ در Icon و Label می‌ماند
Navbar با عنوان Hamburger تداخل	نشود. H1 در نظر گرفته شد تا وارد محدوده Navbar در Hamburger برای Divider فضای ثابت و
فونت‌های بسیار کوچک و پراکنده	ها در فایل مرکزی تعریف شدند؛ تغییر فونت یا مقیاس متن از یک محل ممکن شد. Typography Token
Access Token نشست کاربر پس از انقضای	جدید پیاده‌سازی شد Access Token برای دریافت Axios Interceptor و Refresh Token
relation refresh_tokens does not exist در Seed خطای	برای ساخت جدول‌ها، سپس اجرای Alembic Migration ترتیب راه‌اندازی اصلاح شد: ابتدا Seed/Migration Data.
CORS React و FastAPI بین	باید فقط دامنه واقعی Production تنظیم شد و در CORSMiddleware در React توسعه Origin مجاز باشد

۱۲ راه‌اندازی و اجرای پروژه

Backend و PostgreSQL

به صورت زیر است Backend ترتیب صحیح راه‌اندازی، env. پس از ایجاد پایگاه داده و تنظیم فایل

```
python -m alembic upgrade head
python scripts/migrate_json_to_postgres.py --reset
python -m uvicorn app.main:app --reload
```

آدرس‌های مهم در حالت توسعه

آدرس	خدمت
http://127.0.0.1:8000	FastAPI
http://127.0.0.1:8000/docs	Swagger
http://127.0.0.1:8000/health	Health Check
http://127.0.0.1:8000/api/v1	API Base

Frontend

```
cd frontend
npm install
npm run dev
```

FastAPI به Axios قرار گیرد. سپس رابط کاربری از طریق API Backend برابر آدرس VITE_API_URL در فایل محیطی فرانت باید متصل می‌شود.

Docker اجرای

می‌توانند به صورت یکپارچه اجرا شوند. این روش برای PostgreSQL و API سرویس، Docker Compose در صورت استفاده از یکسان سازی محیط توسعه و استقرار مناسب است.

۱۳ آزمون و ارزیابی

استفاده شده است. تست‌ها باید حداقل مسیرهای احراز هویت، محصولات، سفارش‌ها، داشبورد، HTTPX و Pytest از Backend برای Loading/Error State، نمایش، Route Guard، نیز بررسی Frontend را پوشش دهند. در Pagination فیلترها و اهمیت دارد UI بودن و عملیات اصلی Responsive

- Login و Token Refresh تست.
- تست دسترسی نقش‌ها و پاسخ 401/403
- محصولات و مشتریان CRUD تست
- سفارش‌ها Filter و Pagination تست
- Sales Trend و Dashboard های KPI تست
- خالی Database روی یک Seed پایگاه داده و Migration تست
- موبایل Drawer و Responsive Sidebar تست

۱۴ توسعه‌های آینده

- Request در فرانت برای مخفی کردن عملیات غیرمجاز پیش از ارسال Role-Based UI تکمیل
- Production در نسخه HttpOnly Secure Cookie به Refresh Token انتقال
- مدیریت برخی داده‌های پرمصرف Cache برای Redis استفاده از
- فروش و پیش‌بینی تقاضای محصول Forecasting
- تشخیص ناهنجاری در فروش و هشدار هوشمند
- برای خروجی‌های سنگین Background Job و PDF/Excel گزارش‌های پیشرفته
- در صورت نیاز WebSocket با Real-Time Analytics

- برای ثبت عملیات مهم کاربران Audit Log افزودن
- HTTPS و Nginx، Docker با Production استقرار

۱۵ نتیجه‌گیری

قابل توسعه تبدیل شده است. فرانت‌اند Full-Stack به یک معماری Fake Data سامانه تحلیل داده‌های فروش از یک نمونه اولیه مبتنی بر SQLAlchemy با استفاده از Backend دریافت می‌کند و FastAPI های نسخه‌بندی‌شده API اطلاعات را از طریق React پروژه را، Role-Based و ساختار JWT احراز هویت، Alembic های Migration، متصل است. وجود لایه‌های مستقل PostgreSQL برای توسعه‌های بعدی آماده می‌کند.

نیز Backend بودن و سرعت بارگذاری تمرکز دارد. در سمت Responsive، طراحی رابط کاربری بر خوانایی داده، تجربه کاربری مدیریتی جداسازی مسئولیت‌ها و استفاده از پایگاه داده رابطه‌ای باعث شده است که پروژه علاوه بر نیازهای آموزشی و دانشگاهی، ساختاری نزدیک به یک داشته باشد SaaS محصول واقعی.

در مجموع، پروژه نیازهای اصلی شامل نمایش شاخص‌های فروش، تحلیل داده، مدیریت محصولات و مشتریان، سفارش‌ها، گزارش‌ها، جستجو، اعلان‌ها، احراز هویت و ذخیره پایدار داده را پوشش می‌دهد و پایه مناسبی برای امکانات هوشمند و تحلیلی پیشرفته در نسخه‌های آینده فراهم می‌کند.

پیوست : خلاصه ساختار پروژه

```
sales-analytics/
├── frontend/
│   ├── src/
│   │   ├── app/
│   │   ├── components/
│   │   ├── layouts/
│   │   ├── pages/
│   │   ├── context/
│   │   ├── services/
│   │   ├── types/
│   │   └── styles/
│   └── package.json
└── backend/
    ├── app/
    │   ├── api/v1/
    │   ├── core/
    │   ├── database/
    │   ├── models/
    │   ├── schemas/
    │   ├── repositories/
    │   ├── services/
    │   └── tests/
    └── alembic/
```

```
|— scripts/  
|— database/  
|— requirements.txt  
└— docker-compose.yml
```